Chaire Internationale de Physique Mathématique et Applications
CIPMA - Chaire UNESCO

Cours : **Introduction aux Séries Temporelles**

Master 2 Statistique Appliquée aux Vivants

Application des modèles ARIMA et LSTM sous R et Python sur les données de la production mensuelle d'électricité au USA du 01/1973 à 2005

Rédigé par :

AWANDE Carmel

AYONOU Antoine

GANDJI Edmond

Enseignant:

Dr Nestor BOKOSSA

Contents

Introduction **3**

1 Le modèle ARIMA sous R **3**

1.1 Visualisation des données 3

1.1.1 Visualisation de la série 3

1.1.2 Production moyenne d'électricité 4

1.2 Décomposition de la série en tendance et saisonnalité 4

1.3 ACf et Pacf de la série 5

1.4 Vérification de la stationnarité 6

1.5 Spécification du modèle ARIMA 9

1.6 Prédiction 9

1.7 Validation du modèle 11

1.7.1 Autorrécalition 11

1.7.2 Homoscédasticité 12

2 Le modèle LSTM sous Python **12**

2.1 Bref aperçu du modèle LSTM 12

2.2 Visualisation et traitement des données 13

2.3 Création des séquences temporelles 13

2.4 Division des données 14

2.5 Création et entraînement du modèle LST 14

2.6 Evaluation du modèle 14

Conclusion **16**

Introduction

Les séries temporelles sont des données qui représentent des observations collectées à des intervalles réguliers dans le temps. Elles sont utilisées pour étudier et analyser les tendances, les motifs et les comportements récurrents dans les données temporelles. Le processus ARIMA (AutoRegressive Integrated Moving Average) est un modèle statistique couramment utilisé pour analyser et prévoir les séries temporelles. Il combine trois composantes principales : l'autorégression (AR), la différenciation (I) et la moyenne mobile (MA). L'autorégression examine la relation entre les valeurs passées et la valeur actuelle de la série temporelle, la différenciation est utilisée pour rendre la série temporelle stationnaire en éliminant les tendances, et la moyenne mobile prend en compte les erreurs passées pour prévoir la valeur actuelle. Les LSTM (Long Short-Term Memory) sont des réseaux neuronaux récurrents (RNN) particulièrement adaptés à l'analyse des séries temporelles. Les LSTM sont capables d'apprendre les dépendances à long terme dans les données, ce qui les rend efficaces pour prédire les valeurs futures des séries temporelles. Ils sont couramment utilisés dans des domaines tels que la reconnaissance vocale, le traitement du langage naturel et la prévision des séries temporelles.

Dans le présent travail les modèles ARIMA (sous R) et LSTM (sous python) seront appliqués aux donnée de la production mensuelle d'électricité au USA du 01/1973 à 2005.

1 Le modèle ARIMA sous R

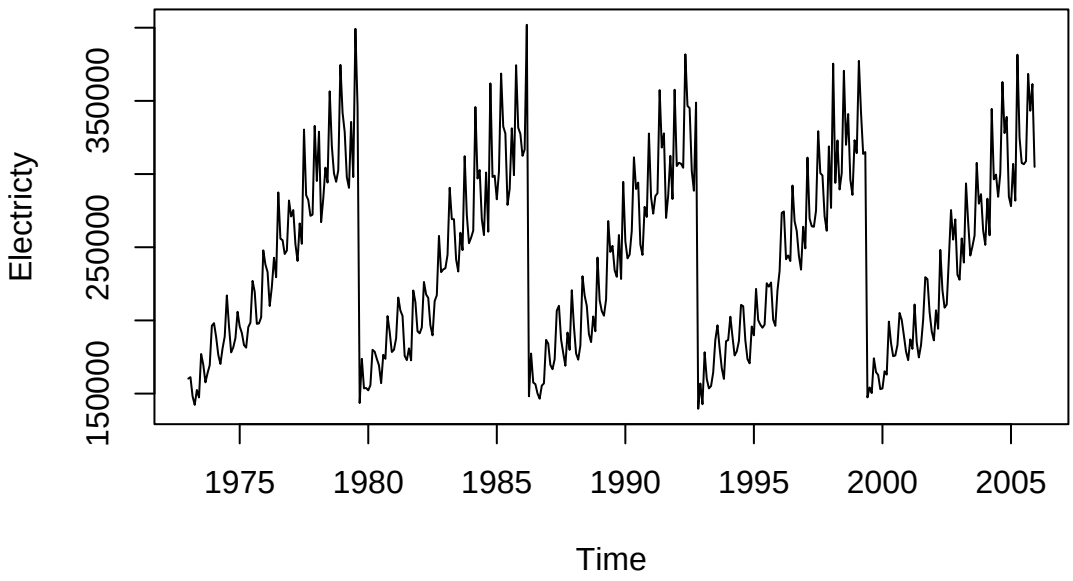
```r
# Chargement des packages
library(readxl)
library(tseries)
library(forecast)
library(lmtest)
```

1.1 Visualisation des données

```r
# Imporation de la base
usElectricity <- read_excel("C:/Users/HP/Downloads/usElectricity.xlsx")
# Création d'un objet de type série temporelle
serie <- ts (usElectricity,start=c(1973,1),end=c(2005,12),frequency=12)
```

1.1.1 Visualisation de la série

```r
plot.ts(serie)
```



La figure présente l'évolution de la production mensuelle d'électricité aux USA.

Après visualisation du graphique, on remarque la production d'électricité varie de 150000 à environ 350000 millions par kW/h sur une période une période de 33 ans.

En examinant le graphique, on peut observer plusieurs tendances. Tout d'abord, il y a une augmentation globale de la production d'électricité au fil du temps, ce qui suggère une croissance continue de la demande énergétique aux États-Unis.Il semble y avoir des variations saisonnières dans la production d'électricité et une légère baisse de la production d'électricité au début des années 2000.

1.1.2 Production moyenne d'électricité

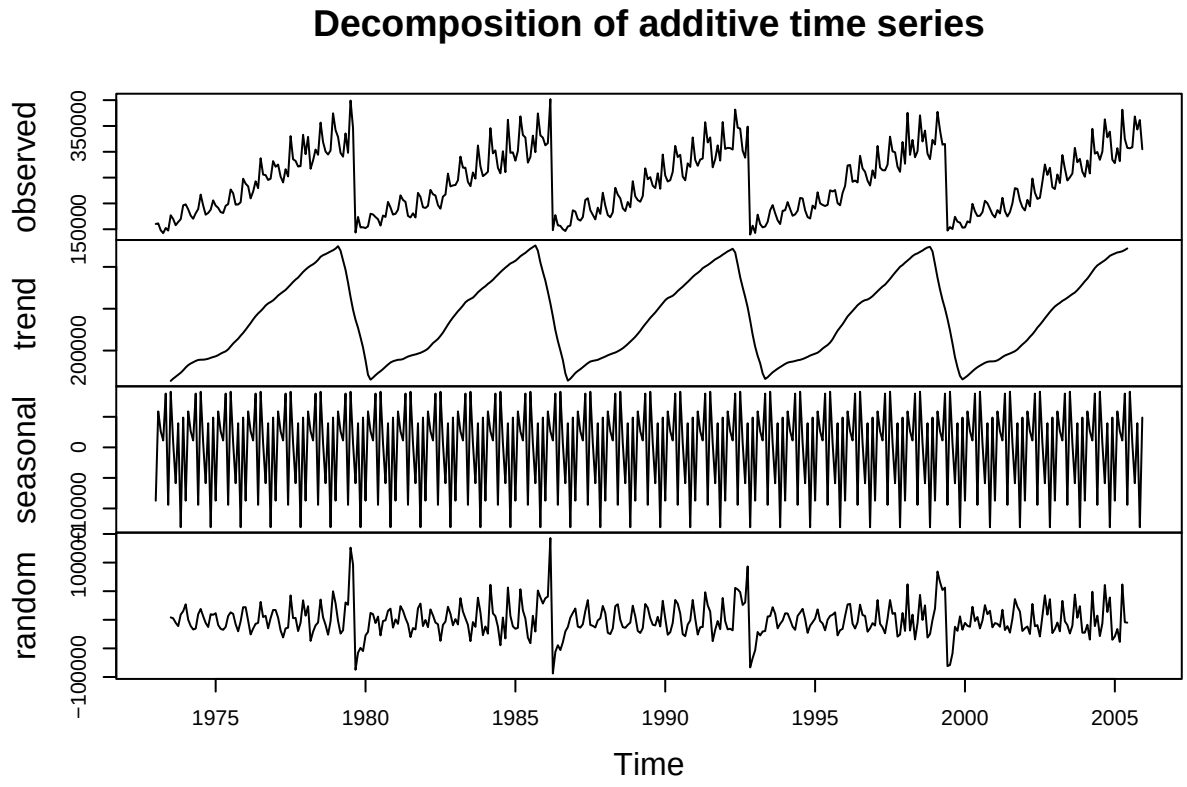
```
mean(serie)
```

```
## [1] 241404.4
```

La production moyenne d'électricté aux USA est 241404.4 millions par kW/h sur une période de 30 ans.

1.2 Décomposition de la série en tendance et saisonnalité

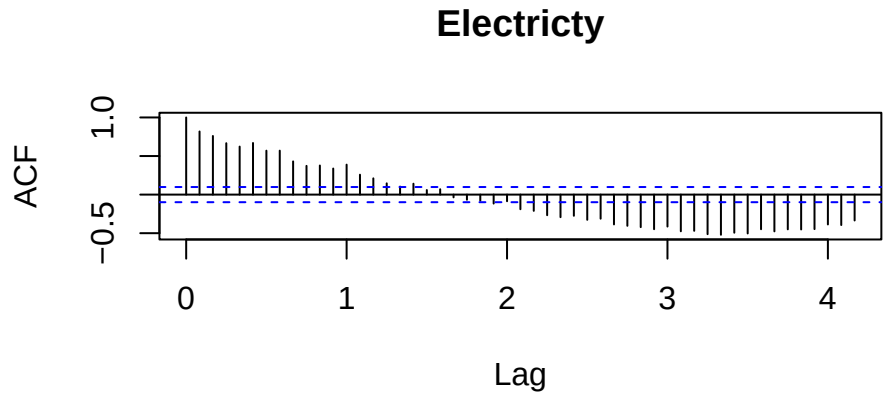
```
dec.m=decompose(serie)
plot(dec.m)
```



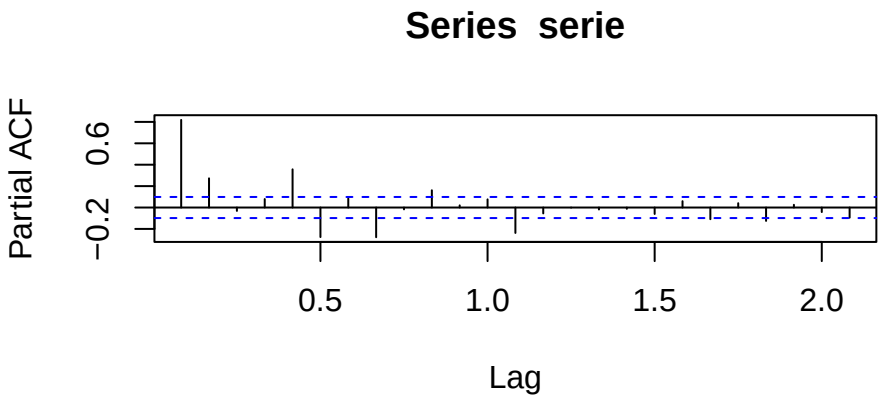
La figure montre la décomposition de la série selon une approche additive. les différentes composantes de la série. Ces composantes sont: la composate aléatoire, la composante saisonière et la tendance. En observant la figure on ne voit pas de tendance dans la série on soupçonne plutôt une saisonnalité.

1.3 ACf et Pacf de la série

```
acf(serie,lag.max = 50)
```



```
pacf(serie)
```



D'après le graphique de l'ACF (fonction d'autocorrélation), il semble qu'il y ait des corrélations significatives entre les observations et leurs retards. Les valeurs de l'ACF dépassent la fourchette de confiance à plusieurs retards, ce qui indique une corrélation statistiquement significative.

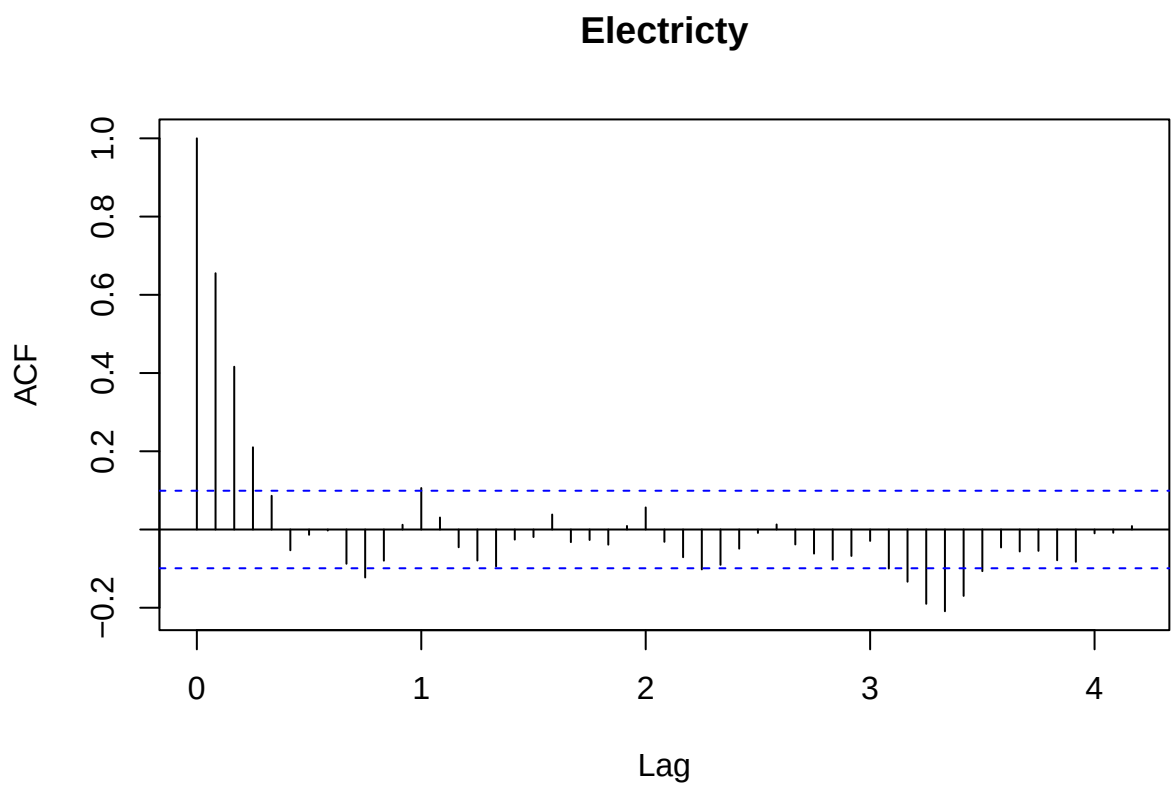
D'après le graphique du PACF (Autocorrélation partielle),nous voyons que la majorité des valeurs situées dans la fourchette, nous soupçonnons donc une stationnarité que nous allons confirmer ou infirmer à l'aide des tests de stationnarité.

1.4 Vérification de la stationnarité

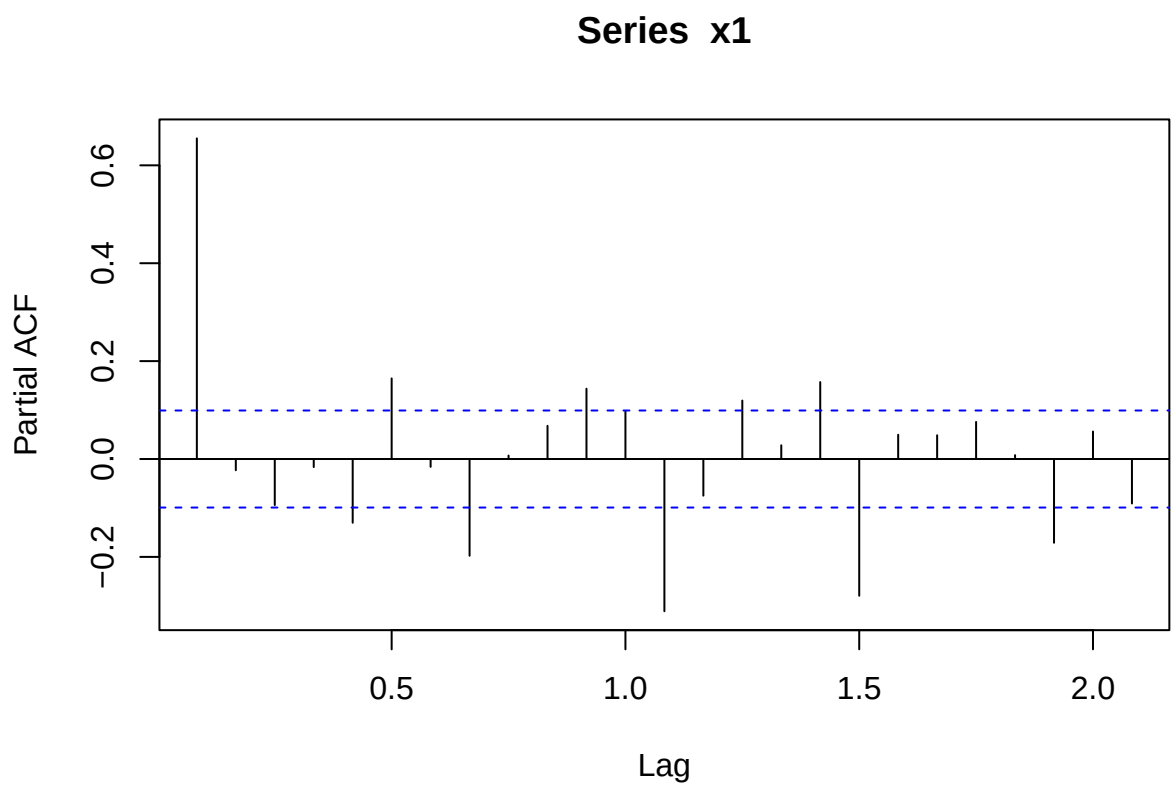
Pour tester la stationarité , nous avons utilisé le Test de Dickey-Fuller augmenté et le Test KPSS Hypothèse du test de Dickey-Fuller Augmenté

```
T=5
x1=diff(serie,lag=T,difference=1)

acf(x1, lag.max = 50)
```




```
pacf(x1)
```



Lorsqu'on effectue une différiation d'ordre 1, on remarque une nette amélioration au niveau de l'ACF. A partir de ce graphique on soupçonne une absence d'autocorrélation que nous allons vérifier à l'aide de test. La série différencié d'ordre 1 est pour la suite de nos analyses.

```
adf=adf.test(x1,alternative = "stationary",k=12)
```

```
## Warning in adf.test(x1, alternative = "stationary", k = 12): p-value smaller
## than printed p-value
```

```
print(adf)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  x1
## Dickey-Fuller = -5.7166, Lag order = 12, p-value = 0.01
## alternative hypothesis: stationary
```

```
kpss.test(x1)
```

```
## Warning in kpss.test(x1): p-value greater than printed p-value
```

```
##
##  KPSS Test for Level Stationarity
```

```
##
## data:  x1
## KPSS Level = 0.046442, Truncation lag parameter = 5, p-value = 0.1
```

$$\begin{cases} H0 : Série \quad non \quad stationnaire \\ H1 : Série \quad stationnaire \end{cases}$$

La p-value associée au Test de Dickey-Fuller augmenté est inférieur à 0.05, donc la série est stationnaire

Hpothèse du Test de KPSS

$$\begin{cases} H0 : Série \quad stationnaire \\ H1 : Série \quad non \quad stationnaire \end{cases}$$

La p-value associée au test de KPSS est supérieure à 0.05, la série est donc stationnaire.

Les deux test de stationnarité effectuée confirme que la série est stationnaire nous allons passer à la modélisation du processus ARIMA.

1.5 Spécification du modèle ARIMA

```r
fit=auto.arima(x1)
summary(fit)
```

```
## Series: x1
## ARIMA(2,0,3)(1,0,1)[12] with zero mean
##
## Coefficients:
##           ar1     ar2     ma1     ma2      ma3    sar1     sma1
##       -0.0032  0.5079  0.7921  0.0345  -0.2258  0.7514  -0.5101
## s.e.   0.1111  0.0684  0.1081  0.1121   0.0918  0.0752   0.0909
##
## sigma^2 = 1.175e+09:  log likelihood = -4635.78
## AIC=9287.56   AICc=9287.94   BIC=9319.31
##
## Training set error measures:
##                    ME     RMSE      MAE     MPE     MAPE      MASE        ACF1
## Training set 430.6424 33974.71 20819.28 43.2878 280.2729 0.6180072 -0.02722164
```

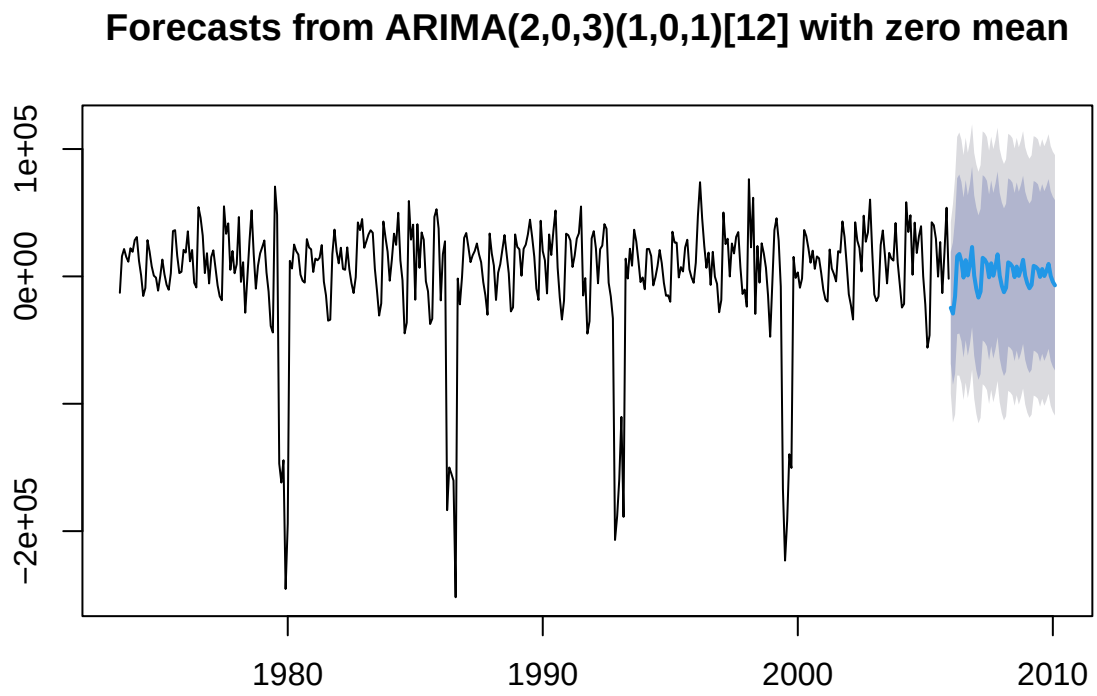
Nous retenons donc ARIMA(2,0,3) comme le meilleur pour la modélisation de nos données(c'est ce modèle qui minimisent l'AIC).

1.6 Prédiction

```r
prediction=forecast(fit,h=50)
prediction
```

```
##          Point Forecast     Lo 80    Hi 80      Lo 95     Hi 95
## Jan 2006    -24650.4851 -68585.89 19284.92  -91843.89  42542.92
## Feb 2006    -29170.8495 -85133.13 26791.43 -114757.77  56416.07
## Mar 2006    -15611.2383 -76392.56 45170.08 -108568.25  77345.77
## Apr 2006     15842.1654 -45413.58 77097.91  -77840.42 109524.75
## May 2006     17595.9469 -44828.46 80020.36  -77873.95 113065.84
## Jun 2006     11596.7838 -50944.77 74138.34  -84052.28 107245.84
## Jul 2006      -908.8171 -63746.57 61928.93  -97010.86  95193.23
## Aug 2006     12764.8840 -50102.31 75632.07  -83382.19 108911.96
## Sep 2006       763.4484 -62179.58 63706.48  -95499.61  97026.51
## Oct 2006      9720.8702 -53229.59 72671.33  -86553.55 105995.29
## Nov 2006     23084.9003 -39885.03 86054.83  -73219.30 119389.10
## Dec 2006       593.7915 -62378.02 63565.60  -95713.28  96900.86
## Jan 2007     -9885.6538 -73880.25 54108.95 -107756.95  87985.64
## Feb 2007    -16619.8673 -81190.89 47951.16 -115372.72  82132.99
## Mar 2007    -11870.4054 -76731.36 52990.55 -111066.68  87325.87
## Apr 2007     14596.7820 -50293.75 79487.31  -84644.72 113838.29
## May 2007     13142.7521 -51822.03 78107.53  -86212.30 112497.81
## Jun 2007     10081.9656 -54890.28 75054.21  -89284.51 109448.44
## Jul 2007      -727.6676 -65718.97 64263.64 -100123.29  98667.96
## Aug 2007     10286.8061 -54706.39 75280.00  -89111.71 109685.32
## Sep 2007       548.7322 -64449.36 65546.82  -98857.27  99954.73
## Oct 2007      7657.5879 -57340.98 72656.15  -91749.14 107064.32
## Nov 2007     17333.0418 -47666.78 82332.87  -82075.61 116741.70
## Dec 2007       625.5126 -64374.43 65625.46  -98783.33 100034.35
## Jan 2008     -7436.0278 -72947.62 58075.57 -107627.37  92755.31
## Feb 2008    -12397.6816 -78216.12 53420.76 -113058.30  88262.94
## Mar 2008     -8924.0027 -74889.67 57041.67 -109809.79  91961.78
## Apr 2008     11014.8411 -54965.92 76995.61  -89894.03 111923.71
## May 2008      9873.6952 -56144.83 75892.22  -91092.93 110840.32
## Jun 2008      7599.4736 -58422.87 73621.82  -93372.99 108571.93
## Jul 2008      -548.0154 -66580.06 65484.03 -101535.31 100439.28
## Aug 2008      7741.8311 -58291.18 73774.84  -93246.94 108730.60
## Sep 2008       411.6801 -65623.82 66447.18 -100580.90 101404.26
## Oct 2008      5760.2630 -60275.48 71796.00  -95232.69 106753.21
## Nov 2008     13024.3321 -53012.05 79060.71  -87969.60 114018.26
## Dec 2008       473.1171 -65563.33 66509.56 -100520.91 101467.14
## Jan 2009     -5587.8942 -71908.60 60732.81 -107016.66  95840.87
## Feb 2009     -9314.5073 -75806.40 57177.39 -111005.08  92376.07
## Mar 2009     -6705.9252 -73279.98 59868.13 -108522.16  95110.31
## Apr 2009      8277.7531 -58304.74 74860.24  -93551.38 110106.89
## May 2009      7419.4040 -59184.18 74022.99  -94441.98 109280.79
## Jun 2009      5710.9287 -60894.78 72316.64  -96153.72 107575.58
## Jul 2009      -411.8277 -67022.96 66199.30 -102284.76 101461.10
## Aug 2009      5817.7023 -60793.97 72429.37  -96056.05 107691.46
## Sep 2009       309.3364 -66303.72 66922.40 -101566.55 102185.22
## Oct 2009      4328.5781 -62284.62 70941.77  -97547.51 106204.67
## Nov 2009      9786.9544 -56826.60 76400.51  -92089.69 111663.59
## Dec 2009       355.5707 -66258.02 66969.16 -101521.12 102232.26
## Jan 2010     -4198.9537 -70971.81 62573.90 -106319.22  97921.31
## Feb 2010     -6999.2366 -73868.15 59869.68 -109266.42  95267.94
```

```
plot(prediction)
```



Nous avons prédits la production d'électricité pour les 50 mois suivants Nous observons une décroissance de la production d'électricité mensuelle au fil des années jusqu'à une stabilisation.

1.7 Validation du modèle

1.7.1 Autorrécalition

```
residu1=residuals(fit)

Box.test(fit$residuals, lag=2, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  fit$residuals
## X-squared = 0.64137, df = 2, p-value = 0.7257
```

Nous avons test l'hypothèse d'absence d'autocorrélation à l'aide du test de Box-Ljung

$$\begin{cases} H0 : Absence \quad d'autocorréltion \\ H1 : Présence \quad non \quad d'autocorrélation \end{cases}$$

La p-value associé au test est supérieure à 0.05 donc l'hypothèse d'absence d'autocorrélation est validée

1.7.2 Homoscédasticité

```
temps=time(serie)
bptest(serie~temps)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  serie ~ temps
## BP = 0.21699, df = 1, p-value = 0.6413
```

Pour tester l'hypothèse d'homoscédasticité nous avons utilisé le test de Breusch Pagan dont les hypothèses sont les suivantes

$$\begin{cases} Ho : Homoscédasticité \\ H1 : Hétéroscédasticité \end{cases}$$

La p-value associée au test est supérieur à 0.05, il y a donc d'homosédasticté.

2 Le modèle LSTM sous Python

2.1 Bref aperçu du modèle LSTM

Le réseau à mémoire à long terme (LSTM) est un réseau neuronal récurrent (RNN) qui vise à résoudre le problème du gradient disparaissant présent dans les RNN traditionnels. Sa relative insensibilité à la longueur de l'intervalle est son avantage par rapport aux autres RNN, aux modèles de Markov cachés et aux autres méthodes d'apprentissage de séquences.

Une unité LSTM courante est composée d'une cellule, d'une porte d'entrée, d'une porte de sortie et d'une porte d'oublie. La cellule se souvient des valeurs sur des intervalles de temps arbitraires et les trois portes régulent le flux d'informations entrant et sortant de la cellule. Les portes d'oubli décident des informations à rejeter d'un état antérieur en attribuant à cet état antérieur, comparé à une entrée actuelle, une valeur comprise entre 0 et 1. Une valeur (arrondie) de 1 signifie que l'information est conservée, et une valeur de 0 signifie qu'elle est rejetée. Les portes d'entrée décident quelles nouvelles informations doivent être stockées dans l'état actuel, en utilisant le même système que les portes d'oubli.

Liste des packages nécéssaires

import pandas as pd

import numpy as np

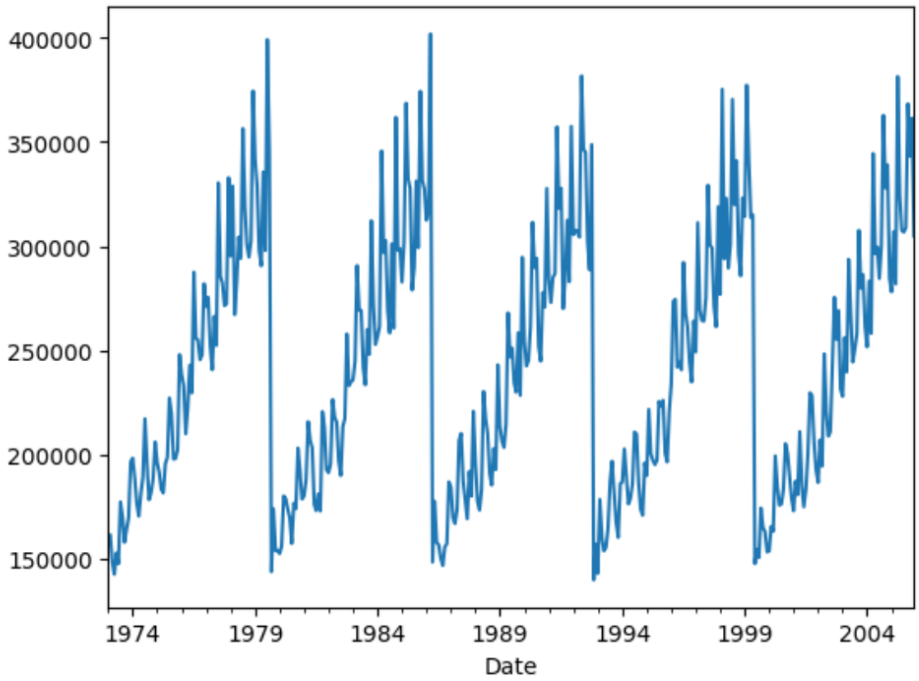
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense

2.2 Visualisation et traitement des données



Nous avons utilisé la fonction **MinMaxScaler** (voir le code ci dessous) pour mettre à l'échelle les valeurs de la série dans la plage de 0 à 1.

Cela est utile pour normaliser les données et les rendre compatibles avec certains algorithmes d'apprentissage automatique.

```
# Prétraiter les données
scaler = MinMaxScaler(feature_range=(0, 1))
df['Electricty'] = scaler.fit_transform(df['Electricty'].values.reshape(-1, 1))
```

2.3 Création des séquences temporelles

Nous avons ici définir une fonction qui permet de créer les séquences temporelles de longeur 12. Cette étape prépare nos données temporelles à l'analyse et à l'apprentissage automatique en ce sens où il permet de capturer les dépendances temporelles, de faciliter la détection de tendances et de motifs dans les données.

```python
# Création des séquences temporelles

def create_sequences(df, seq_length):
    X  = []
    y= []
    for i in range(len(df) - seq_length):
        X.append(df[i:i+seq_length])
        y.append(df[i+seq_length])
    return np.array(X), np.array(y)
```

```python
# Définition de la longueur de la séquence temporelle
seq_length = 12
```

```python
# Création des séquences temporelles
X, y = create_sequences(df['Electricty'].values, seq_length)
```

2.4 Division des données

```python
# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Nous avons divisé ici les données ici en données d'entrainements et de test. 80% des données ont été utilisé pour l'entraînement et 20% pour le test.

2.5 Création et entraînement du modèle LST

```python
# Création et entraînement le modèle LSTM
model = Sequential()
model.add(LSTM(64, input_shape=(seq_length, 1)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=2)
```

Nous avons ici construit, compilé et entraîné le modèle LSTM pour la prédiction en utilisant la bibliothèque Keras.

Le modèle est composé d'une couche LSTM suivie d'une couche dense, et il est entraîné sur les données d'entraînement en utilisant l'optimiseur Adam et la fonction de perte d'erreur quadratique moyenne.

2.6 Evaluation du modèle

Nous avons évalué ici les performances du modèle entraîné sur les ensembles de données d'entraînement et de test en calculant la perte (loss) pour chaque ensemble de données suivant le code ci-après. Cette perte représente en moyenne l'erreur quadratique moyenne.

```python
# Évalution du modèle
train_loss = model.evaluate(X_train, y_train, verbose=0)
test_loss = model.evaluate(X_test, y_test, verbose=0)
print(f'Train Loss: {train_loss}, Test Loss: {test_loss}')
```

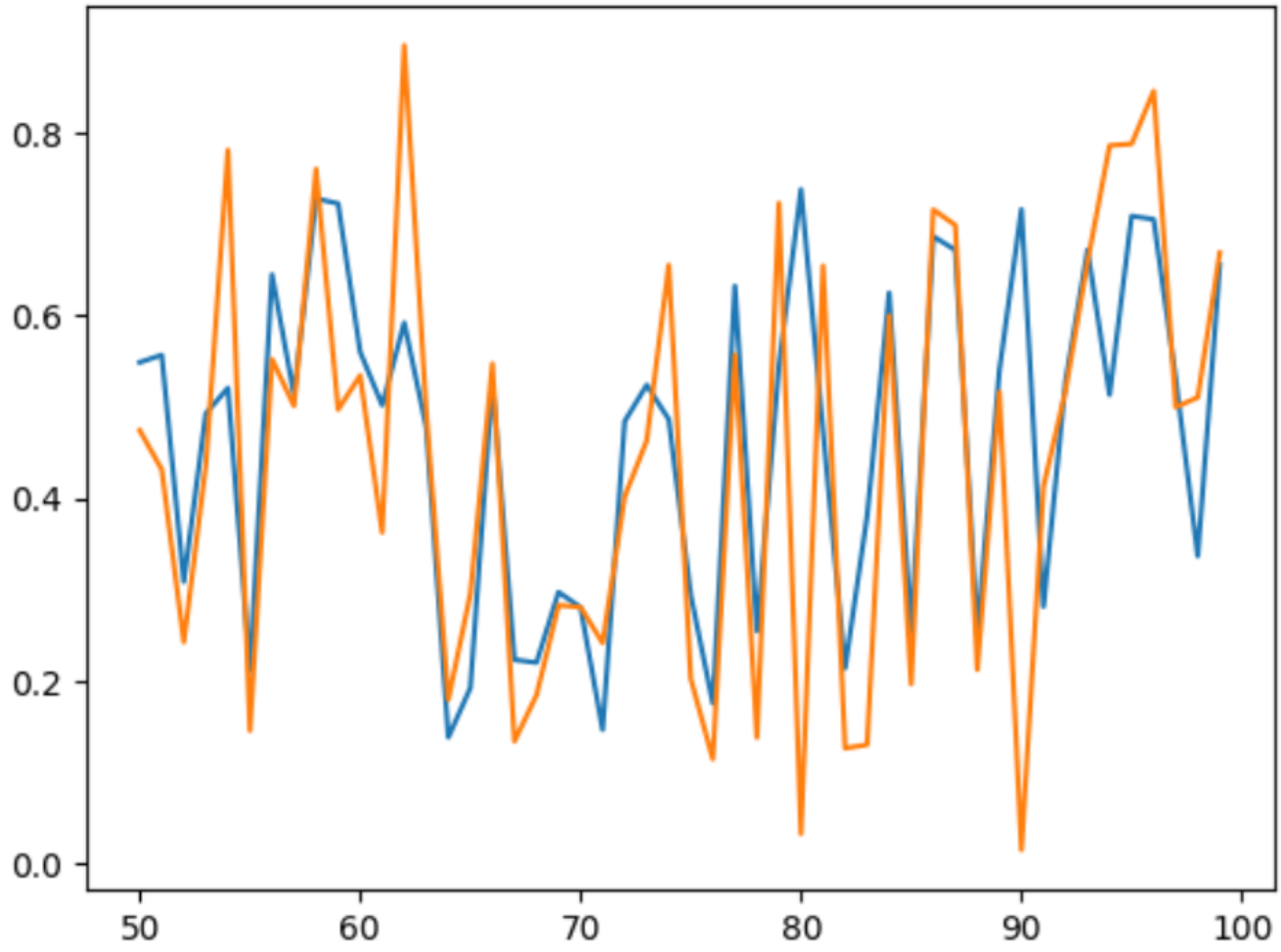
```
Train Loss: 0.016472306102514267, Test Loss: 0.014727260917425156
```

La perte sur l'ensemble d'entraînement est de 0.0165 et La perte sur l'ensemble de test est de 0.0147.

Dans ce cas, les valeurs de perte sont assez proches entre l'ensemble d'entraînement et l'ensemble de test, ce qui suggère que le modèle ne souffre pas de surajustement (overfitting).

Cependant, la perte sur l'ensemble de test est légèrement inférieure à celle sur l'ensemble d'entraînement, ce qui suggère que le modèle pourrait être normalement encore amélioré pour mieux généraliser aux données de test.

Le graphique suivant nous permet de voir l'écart entre les valeurs prédictes (en bleu) du modèles et les données réelles (en orange).



Conclusion

Notre travail porte sur l'application des modèles ARIMA et LSTM sous R et Python sur les données de la production mensuelle d'électricité au USA du 01/1973 à 2005. Au terme de notre travail et après une analyse minutieuse, nous avons identifié que l'ARIMA(2,1,3) est le modèle ARIMA optimal pour notre série temporelle car il a démontré une bonne capacité à capturer les tendances et les variations saisonnières.

Pour le modèle LSTM, nous avons remarqué que la perte sur l'ensemble de test est légèrement inférieure à celle sur l'ensemble d'entraînement, ce qui suggère qu'il n'y a pas overfitting. Cependant on pourrait aller plus loin pour améliorer les performances du modèle LSTM, en explorant des techniques telles que la régularisation, l'ajustement des hypersparamètres.